

Sự kết hợp hoàn hảo giữa số và đồ thị: bàn về hệ ràng buộc sai phân

Feng Wei

Trường Trung học số 1 trực thuộc Đại học Sư phạm Hoa Trung

2006

Nguồn gốc: Sự kết hợp hoàn hảo giữa số và đồ thị: bàn về hệ ràng buộc sai phân

IOI China candidate team papers / 2006 / Feng Wei.doc

Tóm tắt

Khi đối mặt với nhiều bài toán khác nhau, ta thường có thể lập được một mô hình đơn giản theo nghĩa đề bài, nhưng lại không biết xử lý mô hình ấy thế nào. Lúc đó, nên nghĩ tới việc bắc cầu giữa mô hình hiện tại và một mô hình quen thuộc hơn. Bài viết giới thiệu hệ ràng buộc sai phân, một phương pháp liên kết một lớp hệ bất đẳng thức đặc biệt với đồ thị, qua đó biến vấn đề phức tạp thành bài toán đường đi quen thuộc. Trọng tâm là nguyên lý của hệ ràng buộc sai phân, thuật toán Bellman–Ford cần nắm, và hai ví dụ ZJU 1508, ZJU 1420 minh họa cách áp dụng trong thi tin học.

Từ khóa: hệ ràng buộc sai phân; bất đẳng thức; đường đi ngắn nhất một nguồn; chuyển hóa.

1 Bellman–Ford

Trước khi phân tích hệ ràng buộc sai phân, ta nhắc lại thuật toán Bellman–Ford cho bài toán đường đi ngắn nhất một nguồn. Trong hệ ràng buộc sai phân, thuật toán này đóng vai trò công cụ trung tâm.

1.1 Khái quát

Bellman–Ford giải bài toán đường đi ngắn nhất trong trường hợp tổng quát hơn Dijkstra: cạnh có thể mang trọng số âm, và thuật toán còn phát hiện được chu trình âm đi được từ nguồn. Nếu không có chu trình âm như vậy, thuật toán trả về giá trị đúng của đường đi ngắn nhất; nếu có, nó trả về thất bại.

Cho đồ thị có hướng có trọng số

$$G = (V, E),$$

nguồn s , và trọng số cạnh $w(u, v)$. Với mỗi đỉnh v , thuật toán duy trì $d[v]$, tức ước lượng hiện tại của khoảng cách ngắn nhất từ s tới v . Thao tác cơ bản là **nới lỏng** cạnh (u, v) :

$$\text{nếu } d[v] > d[u] + w(u, v) \text{ thì đặt } d[v] = d[u] + w(u, v).$$

Với bài toán đường đi dài nhất trong đồ thị thích hợp, dấu bất đẳng thức có thể đảo chiều.

1.2 Quy trình

Phiên bản cơ bản của Bellman–Ford:

1. Khởi tạo $d[s] = 0$, các $d[v]$ khác là $+\infty$.
2. Lặp $|V| - 1$ lần; mỗi lần duyệt mọi cạnh $(u, v) \in E$ và nối lỏng cạnh đó.
3. Duyệt lại mọi cạnh. Nếu vẫn còn cạnh có thể nối lỏng, tồn tại chu trình âm đi được từ nguồn, trả về false.
4. Nếu không, trả về true; các $d[v]$ là khoảng cách ngắn nhất.

Lý do chỉ cần $|V| - 1$ lượt là mọi đường đi ngắn nhất đơn có nhiều nhất $|V| - 1$ cạnh. Bằng quy nạp theo số cạnh trên đường đi ngắn nhất, sau lượt thứ i , các đỉnh có đường đi ngắn nhất dùng không quá i cạnh đã nhận đúng giá trị. Nếu sau $|V| - 1$ lượt vẫn có thể cải thiện, việc cải thiện ấy phải đi qua một chu trình âm.

1.3 Ví dụ: ZJU 2008 Invitation Cards

Bài *Invitation Cards* cho một đồ thị có hướng có trọng số. Từ trạm trung tâm s , mỗi tình nguyện viên đi tới một trạm được giao, cuối ngày quay về s . Cần cực tiểu tổng chi phí đi và về.

Cách làm là tính tổng đường đi ngắn nhất từ s tới mọi đỉnh, rồi đảo chiều mọi cạnh và tính một lần nữa từ s . Lần thứ hai tương đương với đường đi ngắn nhất từ mọi đỉnh về s trong đồ thị gốc. Tổng hai kết quả là đáp án.

Với dữ liệu lớn, Dijkstra thuần $\mathcal{O}(V^2)$ và Bellman–Ford cơ bản $\mathcal{O}(VE)$ đều quá chậm. Bản gốc so sánh ba cách trên ZJU:

Thuật toán	Thời gian trên toàn bộ dữ liệu
Dijkstra thuần	$> 5s$, TLE
Bellman–Ford dùng hàng đợi	1.46s
Dijkstra + heap	1.24s

Biến thể hàng đợi của Bellman–Ford, thường gọi là SPFA, chỉ đưa vào hàng đợi những đỉnh vừa được cải thiện và có khả năng cải thiện các cạnh đi ra. Trong thực tế nó tránh được nhiều lượt duyệt cạnh vô ích, dù độ phức tạp xấu nhất vẫn cần được đánh giá cẩn thận.

2 Hệ ràng buộc sai phân

2.1 Ví dụ dẫn nhập: ZJU 1508 Interval

Bài *Interval* cho n bộ ba $[a_i, b_i, c_i]$. Mỗi bộ ba nói rằng trong đoạn số nguyên $[a_i, b_i]$ của một dãy, có ít nhất c_i số được chọn. Nếu tồn tại dãy thỏa các yêu cầu, cần tìm độ dài nhỏ nhất của dãy; nếu không tồn tại thì in -1 .

Ta lượng hóa bài toán bằng biến nhị phân t_i , trong đó $t_i = 1$ nếu số i được chọn và 0 nếu không. Đặt tổng tiền tố

$$S_i = t_0 + t_1 + \dots + t_i.$$

Điều kiện “trong $[a, b]$ có ít nhất c số” trở thành

$$S_b - S_{a-1} \geq c.$$

Ngoài ra, do $t_i \in \{0, 1\}$, các tổng tiền tố phải thỏa

$$0 \leq S_i - S_{i-1} \leq 1.$$

Tức là:

$$\begin{aligned} S_i - S_{i-1} &\geq 0, \\ S_{i-1} - S_i &\geq -1. \end{aligned}$$

Các bất đẳng thức này đều có dạng hiệu của hai biến bị chặn bởi một hằng số:

$$X_u - X_v \geq w.$$

Trong Bellman–Ford, sau khi nối lỏng cạnh (v, u) trọng số w cho bài toán đường đi dài nhất, ta muốn bảo đảm

$$d[u] \geq d[v] + w,$$

tức

$$d[u] - d[v] \geq w.$$

Hai dạng hoàn toàn tương ứng. Vì vậy mỗi bất đẳng thức

$$X_u - X_v \geq w$$

được chuyển thành một cạnh có hướng $v \rightarrow u$ trọng số w . Tìm một bộ giá trị thỏa hệ bất đẳng thức trở thành tìm nhãn đỉnh thỏa mọi ràng buộc cạnh. Nếu xuất hiện chu trình làm giá trị phải tăng vô hạn hoặc mâu thuẫn theo dạng đã chọn, hệ không có nghiệm.

Với bài Interval, sau khi xây dựng đồ thị từ các ràng buộc trên, ta chạy biến thể Bellman–Ford phù hợp để tìm giá trị tối ưu của các S_i . Nếu cần in chính dãy, ta khôi phục

$$t_i = S_i - S_{i-1}.$$

Khi $t_i = 1$, số i thuộc dãy được chọn.

2.2 Liên hệ với quy hoạch tuyến tính

Một bài toán quy hoạch tuyến tính tổng quát có dạng: cho ma trận A , véc-tơ b và véc-tơ c , tìm véc-tơ x sao cho

$$Ax \leq b$$

và tối ưu hàm mục tiêu $c^T x$. Nhiều bài toán thực tế có thể được mô tả trong khung này.

Hệ ràng buộc sai phân là một trường hợp đặc biệt của quy hoạch tuyến tính: mỗi hàng của ma trận A chỉ có một hệ số 1, một hệ số -1 , các hệ số khác bằng 0. Vì thế mỗi ràng buộc có dạng

$$x_i - x_j \leq b_k$$

hoặc tương đương sau đổi dấu:

$$x_j - x_i \geq -b_k.$$

Chính cấu trúc hai biến trừ nhau này cho phép chuyển toàn bộ hệ sang đồ thị và giải bằng thuật toán đường đi một nguồn.

Cần lưu ý rằng nghiệm cụ thể Bellman–Ford trả về chỉ là một trong nhiều nghiệm có thể. Các nghiệm khác nhau vẫn có quan hệ với nhau, nhưng bài viết không đi sâu vào vấn đề đó.

2.3 Ví dụ: ZJU 1420 Cashier Employment

Bài *Cashier Employment* yêu cầu bố trí nhân viên thu ngân theo giờ. Gọi $num[i]$ là số người có thể bắt đầu làm việc ở giờ i , $x[i]$ là số người thực sự thuê ở giờ ấy, và $r[i]$ là số người tối thiểu cần có mặt ở giờ i . Mỗi người làm một ca dài 8 giờ.

Ta có

$$0 \leq x[i] \leq num[i].$$

Đặt tổng tiền tố

$$s[i] = x[0] + x[1] + \dots + x[i].$$

Khi đó các ràng buộc số người có mặt có thể viết thành:

$$\begin{aligned} 0 &\leq s[i] - s[i-1] \leq num[i], \\ s[i] - s[i-8] &\geq r[i] & (8 \leq i \leq 23), \\ s[23] + s[i] - s[i+16] &\geq r[i] & (0 \leq i \leq 7). \end{aligned}$$

Nhóm đầu tiên được tách thành hai bất đẳng thức sai phân:

$$\begin{aligned} s[i] - s[i-1] &\geq 0, \\ s[i-1] - s[i] &\geq -num[i]. \end{aligned}$$

Với $8 \leq i \leq 23$, ràng buộc $s[i] - s[i-8] \geq r[i]$ cũng đã đúng dạng. Khó khăn nằm ở nhóm cuối: có ba biến do ca làm việc vòng qua nửa đêm. Quan sát rằng trong đó chỉ có hai biến phụ thuộc i , còn $s[23]$ là tổng số người được thuê trong cả ngày, chính là đại lượng cần cực tiểu. Vì vậy ta có thể liệt kê các giá trị $s[23]$, coi nó là hằng số, rồi kiểm tra hệ ràng buộc sai phân tương ứng có khả thi hay không.

Với mỗi giá trị thử của $s[23]$, đưa mọi bất đẳng thức dạng

$$A - B \geq C$$

thành cạnh $B \rightarrow A$ trọng số C , sau đó chạy Bellman–Ford hoặc SPFA để kiểm tra và tính nhãn. Giá trị khả thi nhỏ nhất của $s[23]$ là đáp án.

3 Kết luận

Khi mô hình ban đầu không rõ cách xử lý, một hướng suy nghĩ quan trọng là chuyển nó sang một mô hình quen thuộc hơn. Hệ ràng buộc sai phân là một ví dụ đẹp: các bất đẳng thức số học có dạng đặc biệt được chuyển thành cạnh có hướng trên đồ thị, rồi giải bằng thuật toán đường đi ngắn nhất hoặc dài nhất một nguồn.

Ý tưởng liên tưởng, phát tán và chuyển hóa như vậy là một công cụ mạnh trong thi tin học. Một khi tìm được cây cầu giữa bài toán mới và kiến thức đã biết, nhiều vấn đề tưởng khó xử lý sẽ trở nên trực tiếp và có hệ thống hơn.

References

- [1] Wang Jiande, Zhou Yongji. *Con đường huy chương vàng: hướng dẫn giải bài thi lập trình trung học*.
- [2] H. Cormen, C. Leiserson, R. Rivest, C. Stein. *Introduction to Algorithms*.
- [3] Zhejiang University Online Judge problem set: ZJU 2008, ZJU 1508, ZJU 1420.